



LAB No. 03

SYSTEM CALLS

NAME : Asim Sultan

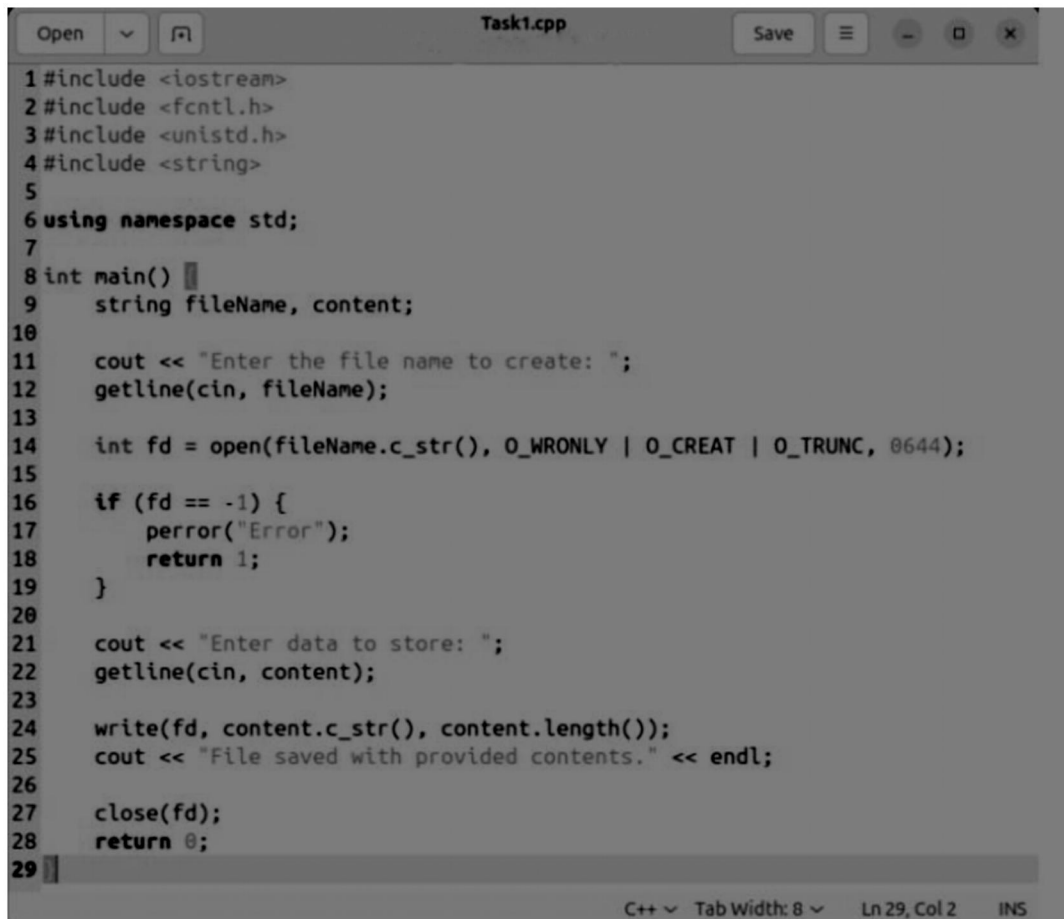
REG # : 9008

LAB TASKS

Task 1

(10 marks)

Write a C++ program which takes **file name** from user and creates the file using file management system call. Later on, ask the user to enter the **data** which he/she wants to store in the file and save the file with provided contents. Demonstrate your program using Makefile utility.



```
1 #include <iostream>
2 #include <fcntl.h>
3 #include <unistd.h>
4 #include <string>
5
6 using namespace std;
7
8 int main() {
9     string fileName, content;
10
11     cout << "Enter the file name to create: ";
12     getline(cin, fileName);
13
14     int fd = open(fileName.c_str(), O_WRONLY | O_CREAT | O_TRUNC, 0644);
15
16     if (fd == -1) {
17         perror("Error");
18         return 1;
19     }
20
21     cout << "Enter data to store: ";
22     getline(cin, content);
23
24     write(fd, content.c_str(), content.length());
25     cout << "File saved with provided contents." << endl;
26
27     close(fd);
28     return 0;
29 }
```

Task 2

(10 marks)

Write a C++ program which takes **names of two files**, i.e. *file_name_1* and *file_name_2*. **Copy** the **contents** of the first **file** into the second **file**. Demonstrate your program using Makefile utility.

```
Open  Task2.cpp  Save  -  □  x
1#include <iostream>
2#include <fcntl.h>
3#include <unistd.h>
4
5using namespace std;
6
7int main() {
8    char f1[100], f2[100], buffer[1024];
9    ssize_t bytes;
10
11    cout << "Enter source file (file_name_1): ";
12    cin >> f1;
13    cout << "Enter destination (file_name_2): ";
14    cin >> f2;
15
16    int src = open(f1, O_RDONLY);
17    int dst = open(f2, O_WRONLY | O_CREAT | O_TRUNC, 0644);
18
19    while ((bytes = read(src, buffer, sizeof(buffer))) > 0) {
20        write(dst, buffer, bytes);
21    }
22
23    cout << "Contents copied successfully." << endl;
24    close(src);
25    close(dst);
26    return 0;
27 }
```

C++ Tab Width: 8 Ln 27, Col 2 INS

Task 3

(10 marks)

Write a C++ program which calls **fork** function 2 times in main function. Print process ID of each child process using **getpid** function. Explore the manual of **getpid** function and use it.

```
Open  Task3.cpp  Save  -  □  X
1 #include <iostream>
2 #include <unistd.h>
3 #include <sys/wait.h>
4
5 using namespace std;
6
7 int main() {
8     // Calling fork 2 times
9     fork();
10    fork();
11
12    // getpid() retrieves the current Process ID
13    cout << "Process ID: " << getpid() << " | Parent ID: " <<
    getpid() << endl;
14
15    // Wait to ensure clean output in terminal
16    wait(NULL);
17    return 0;
18 }
```

C++ ▾ Tab Width: 8 ▾ Ln 7, Col 13 INS